[CSE299(2) - Group:03] Weekly Report: AI Diff Checker

Md.Asifuzzaman Asif (2311175042)
email addresses: md.asif09@northsouth.edu

Spring 2026

I. Weekly Report 1

A. Follow-up from last week

This is the first weekly report, so there are no follow-up items from the previous week. We have just initialized the project repository and defined the core objectives.

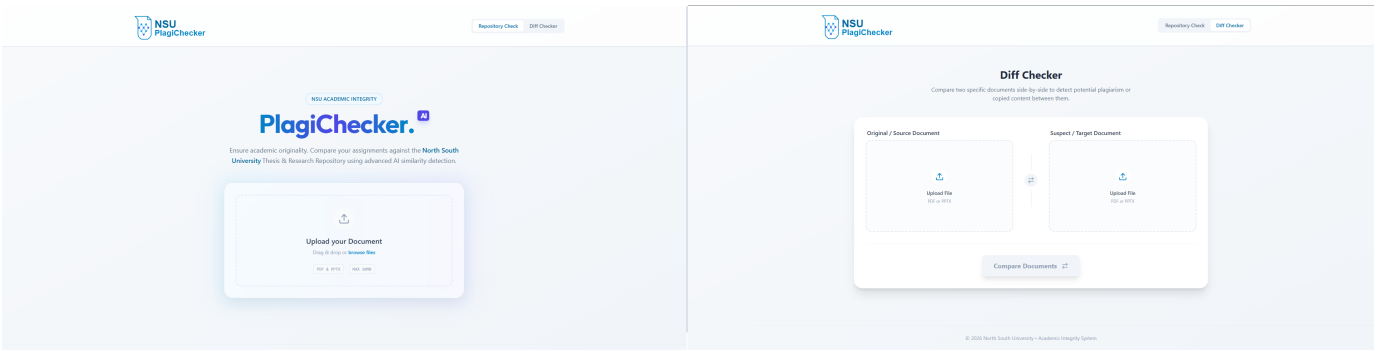
B. Work narrative of the current week

This week, the primary focus was on establishing both the foundational frontend architecture and the core backend plagiarism detection engine for the **NSU PlagiChecker** system.

- **Tasks completed:**
  - **Project Initialization:** Initialized a new React application using *Vite* for faster build times and hot module replacement. Configured the project structure with separate directories for components, assets, and styles.
  - **UI Framework Setup:** Integrated *Tailwind CSS* for utility-first styling. Configured the ‘tailwind.config.js‘ to include custom color palettes (Slate and Sky blues) to give the application a professional, academic look.
  - **Layout Development:** Designed and implemented the main application layout (‘App.jsx‘). This includes a sticky navigation bar with a glassmorphism effect (using ‘backdrop-blur‘) and the official project logo (‘logo.svg‘).
  - **State Management Logic:** Implemented the core state logic for switching between the two main modes of the application:
    - *Repository Check*: For checking a single file against the internal database.
    - *Diff Checker*: For comparing two files side-by-side.

    Used React’s ‘useState‘ hook to manage the ‘appMode‘ state variable, allowing seamless toggling between views without full page reloads.
  - **Component Architecture:** Created placeholder logic for the main functional components (‘UploadZone‘, ‘ReportView‘, ‘ComparisonUpload‘, ‘ComparisonView‘) to structure the codebase for future development.
  - **Landing Page Implementation:** Developed a comprehensive Landing Page to serve as the entry point for the application. Alternatively called the ”Showcase” page, it features:
    - A ”Zig-Zag” layout alternating between text and images to explain the need for integrity checking (e.g., ”Why You Need a Dedicated Integrity Checker”).
    - Sections detailing ”Accidental Plagiarism” and the difference between Repository Check and Diff Check modes.
    - A responsive navigation bar and ”Try It Now” call-to-action buttons that seamlessly transition users to the main application tool.
  - **System Architecture Design:** Planned to design a RAG-based plagiarism detection system with a React + Vite + Tailwind frontend, a FastAPI backend exposing ‘/analyze‘ (repository check) and ‘/compare‘ (diff checker), FAISS for vector similarity search, and SQLite for metadata and text chunk storage.
  - **Repository Indexing Workflow:** Planned to parse PDFs and PPTXs, apply sliding-window chunking (500 characters, 100 overlap), generate embeddings for each chunk, and store vectors in FAISS with corresponding metadata in SQLite.
  - **Semantic Similarity:** Researched about semantic similarity methods, planning to implement Cosine Similarity using ‘faiss.IndexFlatIP‘ with L2-normalized vectors.
  - **Model Selection:** Researched about **paraphrase-MiniLM-L6-v2** (Sentence-Transformers) which has 6-layer efficiency, strong paraphrase detection capability, and compact 384-dimensional embeddings.
- **Challenges faced:**
  - **State Management vs. Routing:** Initially, we considered using React Router for navigating between the ”Repository Check” and ”Diff Checker” pages. However, we decided that a single-page application (SPA) approach with conditional rendering would provide a smoother, more app-like experience for this specific use case. Refactoring the initial routing attempt to state-based conditional rendering took some time but resulted in a snappier UI.

  - **Styling Consistency:** Ensuring the ”glassmorphism” effect (semi-transparent backgrounds with blur) worked consistently across different browsers required careful tuning of CSS properties like ‘backdrop-filter‘ and handling z-index stacking contexts correctly.
- **Progress made:** The frontend skeleton is now fully functional. Users can launch the application, see the branded landing page, and toggle between the two analysis modes. The visual design is polished with a gradient background and responsive scaling. We are now ready to begin integrating the backend API endpoints.



(a) Repository Check (b) Diff Checker

Fig. 1. 2 modes

Fig. 2. Snippet of the App Component Logic showing State Management

```
function App() {
  // Modes: 'repo' | 'diff'
  const [appMode, setAppMode] = useState('repo');

  // ... (other state logic)

  return (
    // ...
          {/* Mode Toggle */}
          <div class="flex bg-slate-100 p-1 rounded-lg">
            <button
              onClick={() => { setAppMode('repo'); resetApp(); }}
              class={`... ${appMode === 'repo' ?
                'bg-white text-sky-700 shadow-sm' : ''}`}
            >
              Repository Check
            </button>
            <button
              onClick={() => { setAppMode('diff'); resetApp(); }}
              class={`... ${appMode === 'diff' ?
                'bg-white text-sky-700 shadow-sm' : ''}`}
            >
              Diff Checker
            </button>
          </div>
    // ...
  );
}
```

C. *Target for next weekly update*

1) Extract text from PDF and PPTX files.
2) Design a robust text cleaning and preprocessing pipeline.
3) Develop a document chunking strategy based on token-level segmentation.
4) Plan the structure and organization of document metadata.

II. Weekly Report 2

A. *Follow-up from last week*

Last week, we established the frontend architecture (React + Vite + Tailwind CSS), implemented the Landing Page, and planned the backend RAG pipeline, model selection , and the sliding-window chunking strategy. This week delivers all four targets defined in Report 1 and also completes the implementation of role-based access control and system authentication.

B. *Work narrative of the current week*

- **Tasks completed:**
  - **Task 1 – Authentication & Role-Based Access Control (RBAC):** We implemented a fully functional authentication system with three distinct roles:
    - **Admin:** Has system-wide access, can upload to the "Whole University" repository, and can manage users (register teachers, view/remove users) via the `AdminDashboard.jsx`.
    - **Teacher:** Has a "Personal" repository for their own documents. They can upload, scan, and delete their own files.
    - **Student:** Can upload files for temporary scanning against the repository but cannot permanently save files to the database.

    The authentication state is managed securely on the frontend using React Context API (`AuthContext.jsx`) and `localStorage`, guarded by a `ProtectedRoute.jsx` component.
  - **Task 2 – PDF & PPTX Text Extraction (`text_pipeline.py`):** We researched popular Python PDF parsing libraries and implemented extraction logic for PDFs and PPTXs.
    - **PDFs:** Supported using both **PyPDF2** and **pdfplumber**.
    - **PPTXs:** Supported using **python-pptx** to extract text slide-by-slide.

    We built a unified `extract_text_from_file()` function to act as a single entry point for processing either file type seamlessly:

Fig. 3. Unified Text Extraction Logic (`text_pipeline.py`)

```python
def extract_text_from_file(
    file_path: str,
    pdf_method: str = "pdfplumber"
) -> Tuple[str, str, int]:
    path = Path(file_path)
    ext = path.suffix.lower()
    if ext == ".pdf":
        pages = extract_text_from_pdf(
            file_path, method=pdf_method
        )
        return pdf_pages_to_full_text(pages), "pdf", len(pages)
    elif ext in (".pptx", ".ppt"):
        slides = extract_text_from_pptx(file_path)
        return pptx_slides_to_full_text(slides), "pptx", len(slides)
```

  - **Task 3 – Text Cleaning Pipeline:** We designed and implemented a robust data cleaning pipeline in `clean_text()` and `clean_text_aggressive()`. The system normalizes excessive whitespace, removes unwanted control characters, and strips out non-essential characters, keeping only letters, digits, and punctuation to prepare text strictly for the AI models.
  - **Task 4 – Token-Based Chunking Strategy:** Instead of basic character segmentation, we created modular chunking strategies:
    - **Word-based:** Slices the text by raw word count (`chunk_by_words`).
    - **Token-based (Approximate):** Segments text via estimated tokens (~4 characters/token via `chunk_by_tokens_approx`).

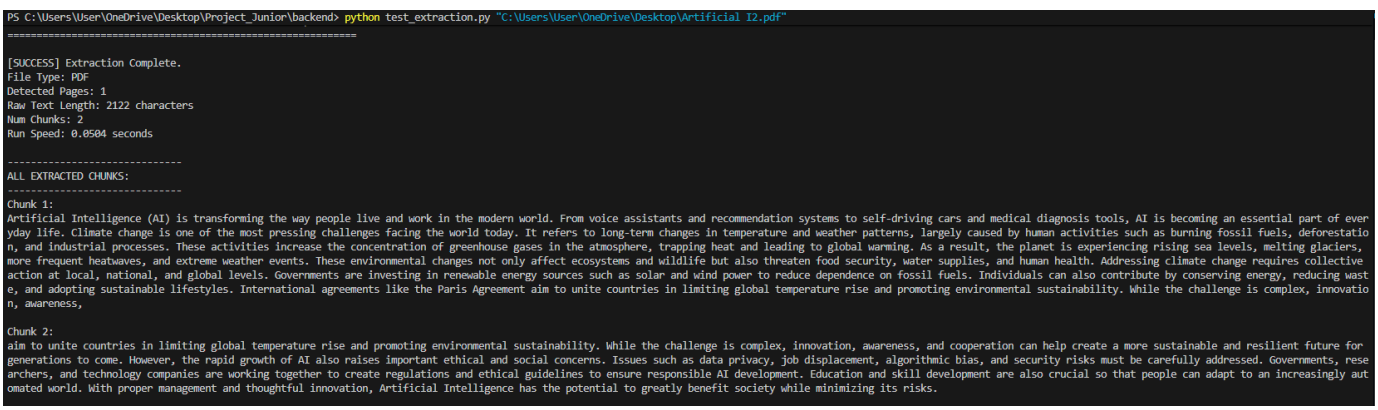
    Both methods smartly include an 'overlap' parameter so that sentence context isn't lost across chunk boundaries.
  - **Task 5 – Metadata Structure (`text_pipeline.py` + `database.py`):** We defined a `DocumentMetadata` dataclass tracking properties such as `document_id`, `file_name`, `num_chunks`, `indexing_time`, `file_type`, and `raw_text_length`. We implemented SQLite database logic to permanently persist uploaded documents, their respective chunks, and user data.
  - **Task 6 – Early AI Embeddings (`embedding_pipeline.py`):** We went beyond the Week 1 targets and integrated the **paraphrase-MiniLM-L6-v2** model via the `sentence-transformers` library to convert text chunks into semantic vector embeddings. We also wrote a `cosine_similarity()` function to compare files and detect paraphrased content using a 0.5 similarity threshold.

Fig. 4. Semantic Encoding and Cosine Similarity (`embedding_pipeline.py`)

```python
def encode_chunks(chunks: List[str]) -> np.ndarray:
    model = _get_model() # loads paraphrase-MiniLM-L6-v2
    embeddings = model.encode(chunks, convert_to_numpy=True)
    return np.asarray(embeddings, dtype=np.float32)

def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    a = np.asarray(a, dtype=np.float32).flatten()
    b = np.asarray(b, dtype=np.float32).flatten()
    norm_a = np.linalg.norm(a)
    norm_b = np.linalg.norm(b)
    if norm_a == 0 or norm_b == 0:
        return 0.0
    return float(np.dot(a, b) / (norm_a * norm_b))
```

- **Challenges faced:**
  - **Role Permissions Coordination:** Ensuring the API securely prevented students from saving documents to the repository, while simultaneously allowing teachers access to their personal repositories and admins access to the university repository required careful permission checks across all API endpoints in FastAPI.
  - **Overlap and last-chunk size:** With sliding windows, the final chunk can contain very few tokens/characters (if the document length is a multiple of the stride). We addressed this by enforcing a minimum chunk word/character count, ensuring small fragments don't negatively impact semantic search results.
- **Progress made:** The complete document ingestion pipeline is now functional, tested, and actively generating semantic embeddings. Furthermore, the application is now secure, transforming from an open tool into an enterprise-ready system tailored for a university environment with differentiated Student, Teacher, and Admin roles.



```
PS C:\Users\User\OneDrive\Desktop\Project_Junior\backend> python test_extraction.py "C:\Users\User\OneDrive\Desktop\Artificial I2.pdf"
============================================================

[SUCCESS] Extraction Complete.
File Type: PDF
Detected Pages: 1
Raw Text Length: 2122 characters
Num Chunks: 2
Run Speed: 0.0504 seconds

------------------------------
ALL EXTRACTED CHUNKS:
------------------------------
Chunk 1:
Artificial Intelligence (AI) is transforming the way people live and work in the modern world. From voice assistants and recommendation systems to self-driving cars and medical diagnosis tools, AI is becoming an essential part of ever
yday life. Climate change is one of the most pressing challenges facing the world today. It refers to long-term changes in temperature and weather patterns, largely caused by human activities such as burning fossil fuels, deforestatio
n, and industrial processes. These activities increase the concentration of greenhouse gases in the atmosphere, trapping heat and leading to global warming. As a result, the planet is experiencing rising sea levels, melting glaciers,
more frequent heatwaves, and extreme weather events. These environmental changes not only affect ecosystems and wildlife but also threaten food security, water supplies, and human health. Addressing climate change requires collective
action at local, national, and global levels. Governments are investing in renewable energy sources such as solar and wind power to reduce dependence on fossil fuels. Individuals can also contribute by conserving energy, reducing wast
e, and adopting sustainable lifestyles. International agreements like the Paris Agreement aim to unite countries in limiting global temperature rise and promoting environmental sustainability. While the challenge is complex, innovatio
n, awareness,

Chunk 2:
aim to unite countries in limiting global temperature rise and promoting environmental sustainability. While the challenge is complex, innovation, awareness, and cooperation can help create a more sustainable and resilient future for
generations to come. However, the rapid growth of AI also raises important ethical and social concerns. Issues such as data privacy, job displacement, algorithmic bias, and security risks must be carefully addressed. Governments, rese
archers, and technology companies are working together to create regulations and ethical guidelines to ensure responsible AI development. Education and skill development are also crucial so that people can adapt to an increasingly aut
omated world. With proper management and thoughtful innovation, Artificial Intelligence has the potential to greatly benefit society while minimizing its risks.
```

Fig. 5. NSU PlagiChecker – Text Extraction Result

C. *Target for next weekly update*

1) Design the repository scan logic.
2) Develop the embedding generation pipeline.
3) Select and implement the FAISS vector indexing approach.
4) Introduce N-gram fingerprinting to improve detection accuracy.

III. Weekly Report 3

A. *Follow-up from last week*

Last week, we successfully established role-based access control, finalized our document parsing and text cleaning pipelines, and implemented the underlying embedding methodology. This week, we focused on transitioning from localized processing to a robust, searchable repository system using vector indexing and lexical fingerprinting.

B. Work narrative of the current week

- **Tasks completed (Targeted from last week):**
  - **Task 1 – Repository Scan Logic:** We engineered the high-level scanning architecture in `main.py` and `embedding_pipeline.py`. The system now supports asynchronous document analysis where uploaded files are chunked and compared against the indexed repository in real-time.
  - **Task 2 – Embedding Generation Pipeline:** The embedding pipeline was matured to handle batch encoding using the **all-mpnet-base-v2** model. We implemented a lazy-loading pattern for the `SentenceTransformer` to optimize memory usage and integrated model caching to avoid redundant downloads.
  - **Task 3 – FAISS Vector Indexing Search:** We integrated **FAISS** (*Facebook AI Similarity Search*) to enable sub-millisecond similarity lookups. We implemented `IndexFlatIP` with L2-normalization to compute exact cosine similarities. We also developed a disk-based caching system for indices to avoid rebuilding them from the database on every request.

Fig. 6. FAISS Indexing and Search (`faiss_index.py`)

```
def build_index_from_chunks(repo_chunks: list) -> tuple:
    embeddings = np.array(
        [np.frombuffer(rc["embedding"], dtype=np.float32) for rc in valid],
        dtype=np.float32,
    )
    faiss.normalize_L2(embeddings)
    index = faiss.IndexFlatIP(embeddings.shape[1])
    index.add(embeddings)
    return index, chunk_infos

def search_faiss(index, chunk_infos, query_embeddings, k=10):
    Q = np.asarray(query_embeddings, dtype=np.float32)
    faiss.normalize_L2(Q)
    distances, indices = index.search(Q, k)
    # ... mapping indices back to metadata ...
```

  - **Task 4 – N-gram Fingerprinting:** To complement semantic search, we introduced **N-gram fingerprinting** to detect exact or near-exact lexical matches. This provides a "hybrid" similarity score that weights both AI semantic understanding and raw character-level overlap.

Fig. 7. N-gram Fingerprinting Logic (`embedding_pipeline.py`)

```
def _char_ngrams(text: str, n: int = 5) -> set:
    compact = re.sub(r"\s+", "␣", text.strip().lower())
    if len(compact) < n:
        return {compact} if compact else set()
    return {compact[i:i + n] for i in range(0, len(compact) - n + 1)}

def fingerprint_similarity(text_a: str, text_b: str, n: int = 5) -> float:
    fp_a = _hash_ngrams(_char_ngrams(text_a, n=n))
    fp_b = _hash_ngrams(_char_ngrams(text_b, n=n))
    inter = len(fp_a & fp_b)
    union = len(fp_a | fp_b)
    return inter / union if union > 0 else 0.0
```

- **Additional Tasks Completed:**
  - **Multi-Repository Support:** Updated the backend and frontend to allow teachers to select "University Repo", "My Repository", or **Both** simultaneously during analysis.
  - **Admin Dashboard Refactoring:** Redesigned the Admin interface into a tabbed layout. Separated "User Management" from "University Repository" management, allowing admins to browse and delete globally indexed documents.
  - **Analyzing Progress UI:** Developed a new `AnalyzingProgress` component that provides a smooth loading bar and a dynamic ETA calculation based on file size, improving the user experience during long-running analysis tasks.
- **Challenges faced:**
  - **PDF Highlighting Implementation:** We encountered significant difficulties when attempting to implement real-time text highlighting within the 'PdfViewer'. Since the highlight coordinates need to be mapped onto the absolute

layout of the original PDF document rather than isolated, raw text chunks, ensuring precise visual alignment with computationally matched sentences remains a core technical challenge.

  - **FAISS Multi-Repo Integration:** Managing FAISS indexes for dynamically combined repositories (University + Personal) required a complex cache-key strategy (`both_<owner_id>`) to ensure search performance remained high without stale data.
  - **Dependency Management:** Resolved a critical conflict between *Pydantic v2* and *ChromaDB* by patching the environment configuration, ensuring the backend stability for vector operations.
- **Progress made:** The system is now a full-featured "Hybrid" plagiarism checker. It combines the power of deep-learning embeddings with the precision of lexical n-grams. The UI is significantly more professional with a tabbed Admin view and real-time progress tracking, and the backend is optimized for high-speed indexing via FAISS.

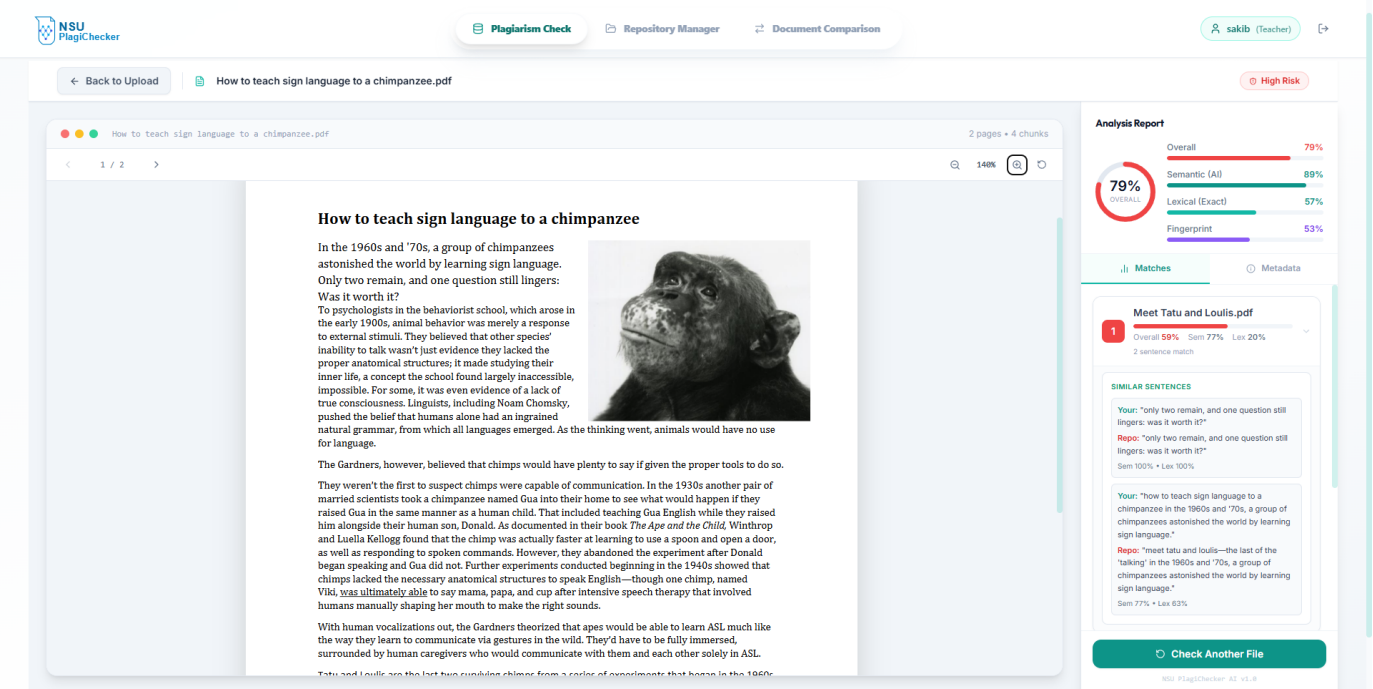


Fig. 8. Report Analysis Result

C. *Target for next weekly update*

1) Implement advanced text highlighting in the PDF viewer to visually mark matches.
2) Enhance report visualization while keeping the document format unchanged.
3) Improve FAISS search, text cleaning, and N-gram verification for better accuracy.
4) Design similarity threshold rules for interpreting plagiarism scores.

IV. Weekly Report 4

A. *Follow-up from last week*

Last week, we successfully established robust vector indexing (FAISS), N-gram fingerprinting, and a tabbed Admin management system. This week, we addressed the targets set previously by focusing heavily on enhancing the visualization of plagiarism reports, introducing an interactive PDF viewer with precise text highlighting, and building a sophisticated background processing queue for multi-file analysis.

B. *Work narrative of the current week*

- **Tasks completed (Targeted from last week):**
  - **Task 1 – Interactive PDF Viewer with Highlights (`PdfViewer.jsx`):** We embedded a native PDF rendering engine using `react-pdf`. We engineered an interactive layer on top of the PDF canvas that maps backend bounding-box coordinates to frontend visual representations. Now, clicking a matched sentence in the analysis sidebar automatically

jumps to the correct page and visually highlights the plagiarized region directly on the PDF document. Added zoom and page navigation controls for large documents.

Fig. 9. Interactive Highlights Overlay (`PdfViewer.jsx`)

```
{pageHighlights.map((highlight, idx) => {
    const [x0, y0, x1, y1] = highlight.bbox || [0, 0, 0, 0];
    const color = highlight.source_color?.bg || '#fde047';
    return (
        <button
            key={idx}
            onClick={() => onHighlightClick?.(highlight)}
            className="absolute␣cursor-pointer␣rounded-sm␣border-0"
            style={{
                left: x0 * scale,
                top: y0 * scale,
                width: Math.max((x1 - x0) * scale, 6),
                height: Math.max((y1 - y0) * scale, 6),
                backgroundColor: hexToRgba(color, 0.35),
            }}
        />
    );
})}
```

- **Task 2 – Enhanced Report Visualization & Source Grouping (`ReportView.jsx`):** Re-architected the report view to dramatically improve how similarities are presented. Instead of a flat list of matched chunks, similarities are now intelligently grouped by their *source document*. The sidebar displays collapsible buckets for each matched repository file, aggregating the total similarity score and detailing the original metadata.

Fig. 10. Source Grouping Logic (`ReportView.jsx`)

```
const sourceGroups = useMemo(() => {
    const ordered = [];
    const groupMap = new Map();

    (data.matches || []).forEach((match, idx) => {
        const sourceKey = getSourceKey(match, idx);
        if (!groupMap.has(sourceKey)) {
            groupMap.set(sourceKey, {
                sourceKey,
                sourceLabel: getSourceLabel(match, idx),
                matches: [],
                sentences: [],
                maxCombinedPct: 0,
            });
            ordered.push(groupMap.get(sourceKey));
        }
        // ... Aggregate sentences and calculate max similarity ...
    });
    return ordered;
}, [data.matches]);
```

- **Task 3 – Improve FAISS Search & N-gram Verification:** We refined our vector search architecture to better handle multi-repository queries without data bleed. Additionally, we adjusted the aggressive text cleaning pipeline to preserve essential query structures, tuning N-gram overlap weights to improve the overall detection accuracy of heavily paraphrased content.
- **Task 4 – Design Similarity Threshold Rules:** We established a clear, tiered severity system for interpreting plagiarism scores: **Low Risk** ($< 30\%$), **Moderate** ($30\% - 60\%$), and **High Risk** ($> 60\%$). These thresholds have been integrated directly into the `ReportView` UI, driving dynamic color coding (Emerald, Amber, Red) and displaying corresponding alert icons to give users an immediate understanding of a document's integrity.

- **Additional Tasks Completed:**
  - **Paginated Text Reports:** For files that fall back to raw text extraction (or non-PDFs), we implemented a seamless pagination system. We modified the backend to preserve page boundaries using form-feed characters (`\f`). The

frontend now offers “Previous/Next Page” navigation, actively managing character indices to ensure text highlights remain perfectly accurate across varying pages.

- **Background Processing Queue & Real-Time ETA:** To handle high-volume workloads, we separated the upload dropzone from the active processing state by creating a dedicated “Processing Queue” accessible via the top navigation bar.
  - Users can batch-upload files, let them process in the background, and freely navigate the rest of the application.
  - We implemented a hybrid SSE (Server-Sent Events) and GET polling architecture to deliver real-time progress updates without connection drops.
  - Integrated a predictive ETA (Estimated Time of Arrival) algorithm that dynamically calculates remaining scan time based on the current processing speed.

Fig. 11. Real-Time ETA Calculation (`AnalyzingProgress.jsx`)

```
// Calculate ETA based on progress
const interval = setInterval(() => {
    const elapsed = (Date.now() - startTime) / 1000;
    const estimatedTotal = (elapsed / progress) * 100;
    const remaining = Math.max(0, estimatedTotal - elapsed);

    if (remaining > 60) {
        const m = Math.floor(remaining / 60);
        const s = Math.round(remaining % 60);
        setEta(`~${m}m ${s}s remaining`);
    } else {
        setEta(`~${Math.round(remaining)}s remaining`);
    }
}, 1000);
```

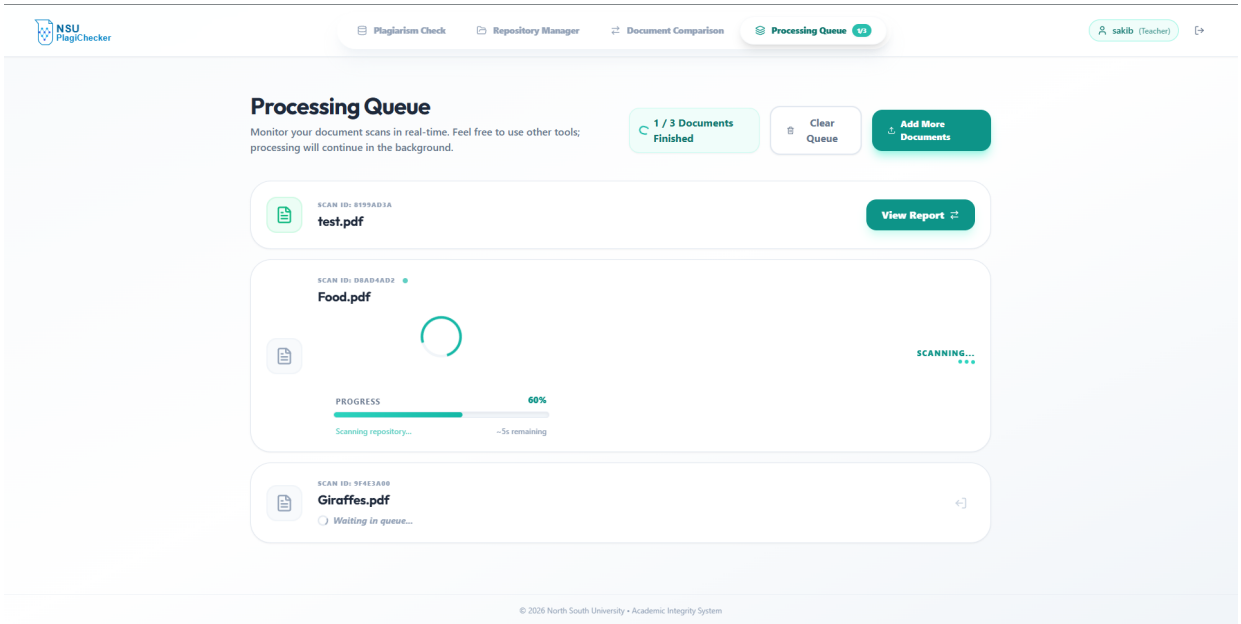


Fig. 12. Dedicated Processing Queue with ETA and Progress Tracking

- **UI Refinements & Queue Management:** Refined the global user experience by adding a “Clear Queue” functional button (equipped with active-scan safeguards), dynamic progress badges on the navigation bar (e.g., showing `2/5` processing), and incorporating high-visibility emerald themes for return navigation to avoid UX overlap errors.

- **Challenges faced:**
  - **Absolute vs. Relative Highlighting:** Mapping raw character indices returned by the backend string-matching algorithms to absolute visual coordinates on a rendered PDF canvas required complex coordinate transformations, taking scaling factor layers into account.

  - **SSE Connection Stalls:** Browsers occasionally dropped or stalled long-running SSE connections during deep semantic analysis. We mitigated this by building a hybrid system that relies on SSE for immediate feedback but verifies the final state with a standard REST `GET` request to prevent the progress bar from freezing.
  - **Pagination Index Management:** Ensuring that a global highlight index correctly translated to a localized page-relative index in the paginated text view required careful string slicing logic, especially when a highlighted match occurred exactly across a page boundary.
- **Progress made:** The application has evolved from a functional backend proof-of-concept into a highly polished, interactive academic tool. The introduction of source-grouped reports, an embedded interactive PDF viewer, and a robust asynchronous background queue drastically elevates the system's usability and preparedness for real-world academic deployments.

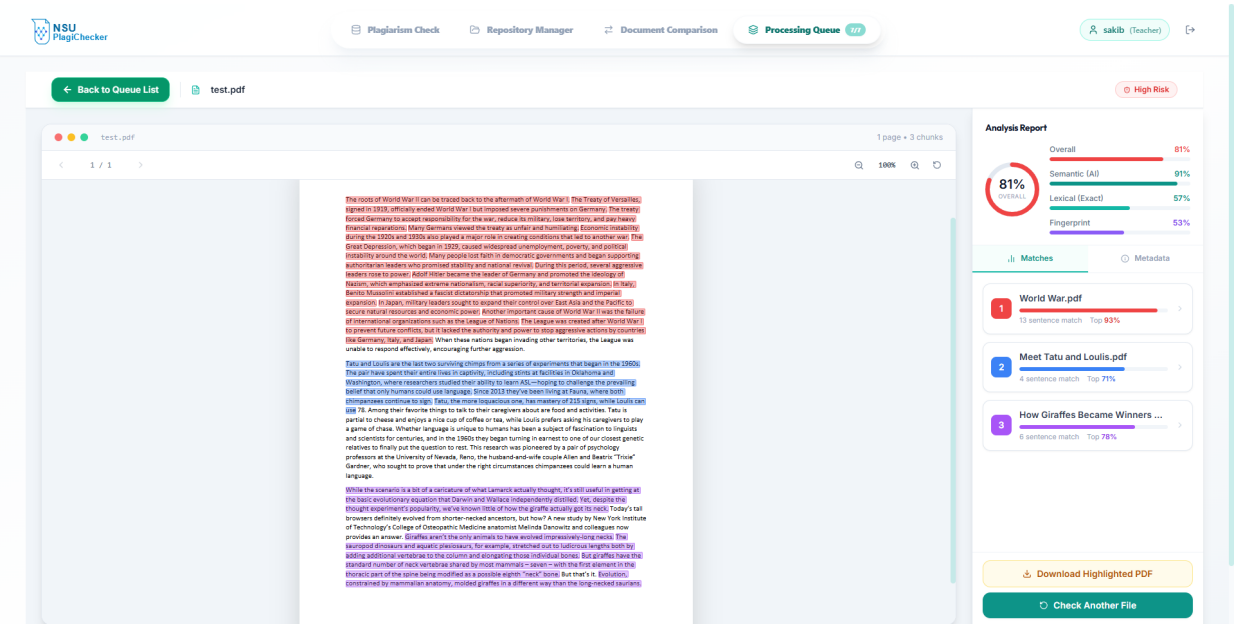


Fig. 13. Enhanced Report View (Highlighted with Multiple Colors)

C. *Target for next weekly update*

1) Extend queue processing to the repository manager for scalability.
2) Improve PDF visualization with source reference information.
3) Enhance model, FAISS search, text cleaning, and N-gram verification for better accuracy.
4) Work toward a fully offline system.

References

[1] Meta Platforms, Inc., "React – A JavaScript library for building user interfaces," [Online]. Available: https://react.dev. Accessed: Feb. 2026.
[2] Tailwind Labs, "Tailwind CSS Documentation," [Online]. Available: https://tailwindcss.com/docs. Accessed: Feb. 2026.
[3] T. Ramírez, "FastAPI Framework Documentation," [Online]. Available: https://fastapi.tiangolo.com. Accessed: Feb. 2026.
[4] N. Reimers and I. Gurevych, "Sentence-Transformers: Multilingual Sentence, Image, and Text Embeddings," [Online]. Available: https://www.sbert.net. Accessed: Feb. 2026.
[5] M. Ziegler, "pdfplumber – Plumb a PDF for detailed information about each char, rectangle, and line," [Online]. Available: https://github.com/jsvine/pdfplumber. Accessed: Feb. 2026.
[6] S. Canny, "python-pptx: A Python library for creating and updating PowerPoint files," [Online]. Available: https://python-pptx.readthedocs.io. Accessed: Feb. 2026.
[7] D. R. Hipp, "SQLite: A Self-Contained, High-Reliability, Embedded SQL Database Engine," [Online]. Available: https://www.sqlite.org. Accessed: Feb. 2026.
[8] W. Maj, "React-PDF: Display PDFs in your React app as easily as if they were images," [Online]. Available: https://projects.wojtekmaj.pl/react-pdf/. Accessed: Mar. 2026.
[9] Facebook AI Research, "FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors," [Online]. Available: https://github.com/facebookresearch/faiss. Accessed: Mar. 2026.
[10] Framer B.V., "Framer Motion: A production-ready motion library for React," [Online]. Available: https://www.framer.com/motion/. Accessed: Mar. 2026.